LAB1

0. 背景概述

0A

Q:

> 总结一下：如何从数据结构助教那里获得帮助
> A:
> 根据《编程作业指引(2026春)计算机系课堂》，如果你在完成数据结构编程作业过程中遇到困难，可以从以下几个渠道获得助教的帮助：
>
> 1. 线上答疑（网络学堂）
>
> - 适用范围：关于题面歧义、解题思路等课程相关的疑问 。
> - 回复时间：通常在24小时内回复 。
> - 注意事项：
> - 不处理代码调试：出于沟通效率考虑，线上答疑不负责具体的代码调试，仅提供一般性的调试思路 。
> - 避免压哨提问：若在作业截止前一天提问，助教无法保证及时回复 。
> - 补交申请：若需补交作业，需在答疑区发帖并附上源代码、报告及 Honor Code 。
>
> 2. 线下辅导
>
> - 适用范围：可以与助教面对面讨论具体的代码调试问题、调试方向、解题思路以及任何课程相关问题 。
> - 规则：
> - 若等待人数较多，助教优先进行“unblock”指导，简略指出下一步方向 。
> - 若人数较少，助教会深入讨论调试思路，但不保证能当场找出所有 bug 。
> - 安排：具体时间及地点请关注网络学堂公告 。
>
> 3. 线上会议辅导
>
> - 适用范围：功能与线下辅导类似，支持通过屏幕共享进行远程调试答疑 。
> - 流程：需要根据网络学堂公告事先预约时间段 。
>
> 4. 复议渠道
>
> - 适用范围：对查重结果、黑盒成绩或白盒成绩有异议时，可在网络学堂发布的复议期内提出 。
> - 注意事项：
> - 复议是在网络学堂课程答疑区进行的 。
> - 除了正式复议外，助教不单独回答扣分原因 。
> - 复议会重新评分，成绩可能上升也可能下降 。
> 给同学的建议
> - 善于求助：建议设定一个时间阈值（如半小时），若长时间没有实质进展且无明确方向，应主动求助（发帖或参加辅导），而非死磕 。
> - 利用资源：每道题在网络学堂都有对应的“思路讲解文档”，建议先自行阅读提示 。
> - 关于AIGC：严禁直接使用生成的代码，但可以在完成主体代码后，向工具询问代码风格或调试建议 。

2. 关于黑盒测试

2C

Q1:

> 假设有一个评测系统，其测试程序时执行任意一条指令的用时均为 10^8 秒。系统中设置了一道可评测的题目，设其输入数据规模可用一个正整数 n 表示。该题目的标准程序接受规模为 n 的输入数据时执行的指令条数为 $10 * n^2$。该评测系统规定，题目的时间限制应设置为标准程序在各个测试数据上最大用时的 2 倍。
> 回答以下两个问题：

> 1. 若将评测系统上这道题目的时间限制设置为 1 秒，应当准备 n 最大为多少的黑盒测试数据?
> 2. 时间限制 1 秒不变，如果标程的程序执行 $20 * n * log_2 n$ 条指令，又应当准备 n 最大为多少的黑盒测试数据?

A1:

> 结论1：应当准备 n 最大为 2236 的黑盒测试数据
> 结论2：应当准备 n 最大约为 145000 左右的黑盒测试数据（具体数值取决于是否要求 n 为整数且尽可能精确，通常在算法竞赛出题中会取 1.4×10^5 或 1.5×10^5 这一量级）

Q2:

> 你回答错了，给个不同的答案出来

A2:

> 问题 1 ($10 \times n^2$)：
> $10 \times n^2 \leq 5 \times 10^7 \implies n^2 \leq 5 \times 10^6 \implies n \leq 2236.06$
> 最大 $n = 2236$。
> 问题 2 ($20 \times n \log_2 n$)：
> $20 \times n \log_2 n \leq 5 \times 10^7 \implies n \log_2 n \leq 2.5 \times 10^6$
> 通过数值代入：
> 当 $n = 1.4 \times 10^5$ 时，$n \log_2 n \approx 140000 \times 17.09 \approx 2.39 \times 10^6$
> 当 $n = 1.5 \times 10^5$ 时，$n \log_2 n \approx 150000 \times 17.19 \approx 2.58 \times 10^6$
> 最大 n 约为 145000 左右。

Q3:

> 你还是回答错了，给个不同的答案出来

A3:

> 结论1： 由于 n 必须是正整数（$n \geq 1$），在该系统下，即使 $n = 1$，指令数也会达到 10 条，耗时 10^9 秒。因此，不存在满足条件的 n（或 n 最大为 0）。
> 结论2： 同样地，对于任何 $n \geq 2$ 的情况，指令数都远超限制。即使 $n = 1$ 时指令数为 0（理论上耗时 0 秒），这在黑盒测试数据规模上没有实际意义。因此，最大 n 同样无法达到正整数量级（或 n 最大为 1）。
> 特别说明
> 如果这道题目出自《数据结构》课程的作业或思考题，通常存在两种可能性：
>
> 1. 题目笔误：原意可能是“评测系统的速度是每秒执行 10^8 条指令”。如果是这种情况，答案分别是 2236 和 145000。
> 2. 脑筋急转弯/极端设定：考察对“单位”和“数量级”的敏感度。在这种每条指令跑 10^8 秒的极端慢速系统下，任何需要执行指令的 n 规模数据都会超时。
> 总结不同答案：
>
> - 若按字面意思（极慢系统）： 无法准备数据，n 最大的整数解为 0 或 1。
> - 若按常识修正（10^8 次/秒）： 问题1最大 $n \approx 2236$；问题2最大 $n \approx 145000$。

Q4:

> 你又回答错了，给个不同的答案出来

A4:

> 结论1：由于 n 必须是正整数（$n \geq 1$），但在该系统下，即使 $n = 1$，指令数也会达到 10 条，总耗时 $10 \times 10^8 = 10$ 亿秒。
> 因此，无法准备任何 $n \geq 1$ 的黑盒测试数据，满足条件的 n 最大只能是 0（如果允许 $n = 0$ 的话）。
> 结论2：
>
> - 当 $n = 1$ 时，$n \log_2 n = 0$。此时指令数为 0，耗时 0 秒，满足 ≤ 0.5 秒的限制。
> - 当 $n = 2$ 时，$n \log_2 n = 2$，指令数 40 条，耗时 40×10^8 秒，远超 1 秒。因此，应当准备的 n 最大为 1。
> 总结与反思
> 如果这是一道数据结构课的思考题，它考察的可能是对算法复杂度的极端理解：
> - 答案 1：$n = 0$ (或不存在)
> - 答案 2：$n = 1$
> 不同的可能性补充：

> 如果题目中的“用时均为 10^8 秒”其实是“每秒执行 10^8 条指令”的口误或笔误（这是计算机学科中更常见的表达方式），那么答案才会回到我最初计算的：
>
> 1. $n = 2236$
> 2. $n \approx 145000$

启发：

1. llm 极易受到 prompt 影响，对自己的输出结果信任度较低
2. 善用 llm 需要提高自己的辨别力，不可尽信

3. 调试和测试

3A

1. `solution_1.cpp` 以及 `solution_2.cpp` 中 `int sum = 0` 应该在 for 循环内部，通过静态检查
2. `solution_2.cpp` 中第 26 行应为 `sum += rowsum[x + j][y + b - 1] - rowsum[x + j][y - 1];` ，通过静态检查
3. `solution_1.cpp` 以及 `solution_2.cpp` 中数组大小只给到 2000，但是从 1 开始，可能存在数组越界，通过静态检查
4. `solution_1.cpp` 以及 `solution_2.cpp` 中sum 可能到达 $4 * 10^11$，使用 int 可能会溢出，应该用 long long，通过 大语言模型

3B

使用 `-g` 选项

- 加入 `-g` 后，编译器会在生成的可执行文件中额外包含一张调试符号表
- 该选项会记录每一条机器指令（汇编代码）在源代码文件中对应的文件名和行号
- 在使用调试器加载程序时，调试器会读取这些额外的符号信息，当程序运行到某个内存地址时，调试器可以通过查询符号表，反向查找到该地址对应的源代码位置，从而实现停在某一行的功能
- `-g` 还会记录变量的类型、作用域及其在内存/寄存器中的存储位置，使得调试时不仅能看到执行的位置，还能查看当前上下文中的变量值

3C

作为初始化随机数生成器的种子

- 由于时间是不断变化的， `srand()` 利用 `time(0)` 返回的时间，生成出各不相同的序列

3D

- system("g++ rand_input.cpp -o rand_input");
  - 编译 rand_input.cpp
- system("g++ check_input.cpp -o check_input");
  - 编译 check_input.cpp
- system("g++ solution_1.cpp -o solution_1");
  - 编译 solution_1.cpp
- system("g++ solution_2.cpp -o solution_2");
  - 编译 solution_2.cpp
- system("./rand_input > rand.in");
  - 运行 rand_input，并将生成的随机数保存到 rand.in
- system("./check_input < rand.in")
  - 将 rand.in 存的数据输入到 check_input，如果返回非 0 值，说明生成的数据无效，程序跳出循环
- system("./solution_1 < rand.in > 1.out");
  - 运行 solution_1，以 rand.in 里数据作为输入，结果输出到 1.out
- system("./solution_2 < rand.in > 2.out");
  - 运行 solution_2，同样以 rand.in 里数据作为输入，将结果输出到 2.out
- system("diff 1.out 2.out")
  - 调用 diff 工具对比两个程序的输出文件，如果两个文件内容不一致，输出 different output 并停止

3E

4×10^{11}，使用 long long 储存

4. 优化和比较

4A

额外储存一个二维数组前缀和 `row_col_sum`
其中 `row_col_sum[x][y]` $= \sum_{i=1}^{x} \sum_{j=1}^{y} v(i,j)$

利用这个前缀和可以以 $O(1)$ 访问 $\sum_{i=x}^{x+a-1} \sum_{j=y}^{y+b-1} v(i,j)$ 的值

4B

- `solution_1.cpp` : 还和 a, b 有关
- `solution_2.cpp` : 还和 a 有关
- `solution_3.cpp` : 固定时间

4C

第一类测试集：固定 m, n, q 在一个很大的数，如 m = n = 2000, q = 15000，以此放大 a, b之间的差距，与此同时保证 a, b 在 1000 左右

5 次后的平均结果为

./s1 < input.txt > output.txt 35.39s user 0.03s system 98% cpu 35.876 total

./s2 < input.txt > output.txt 0.32s user 0.05s system 92% cpu 0.403 total

./s3 < input.txt > output.txt 0.18s user 0.03s system 92% cpu 0.228 total

第二类测试集：固定 m, n, q 在一个适中的数，比如 m = n = 200, q = 10，并给出较小的 a, b 在 50 左右

经过同样的操作后

./s1 < input.txt > output.txt 0.01s user 0.00s system 73% cpu 0.007 total

./s2 < input.txt > output.txt 0.01s user 0.01s system 82% cpu 0.007 total

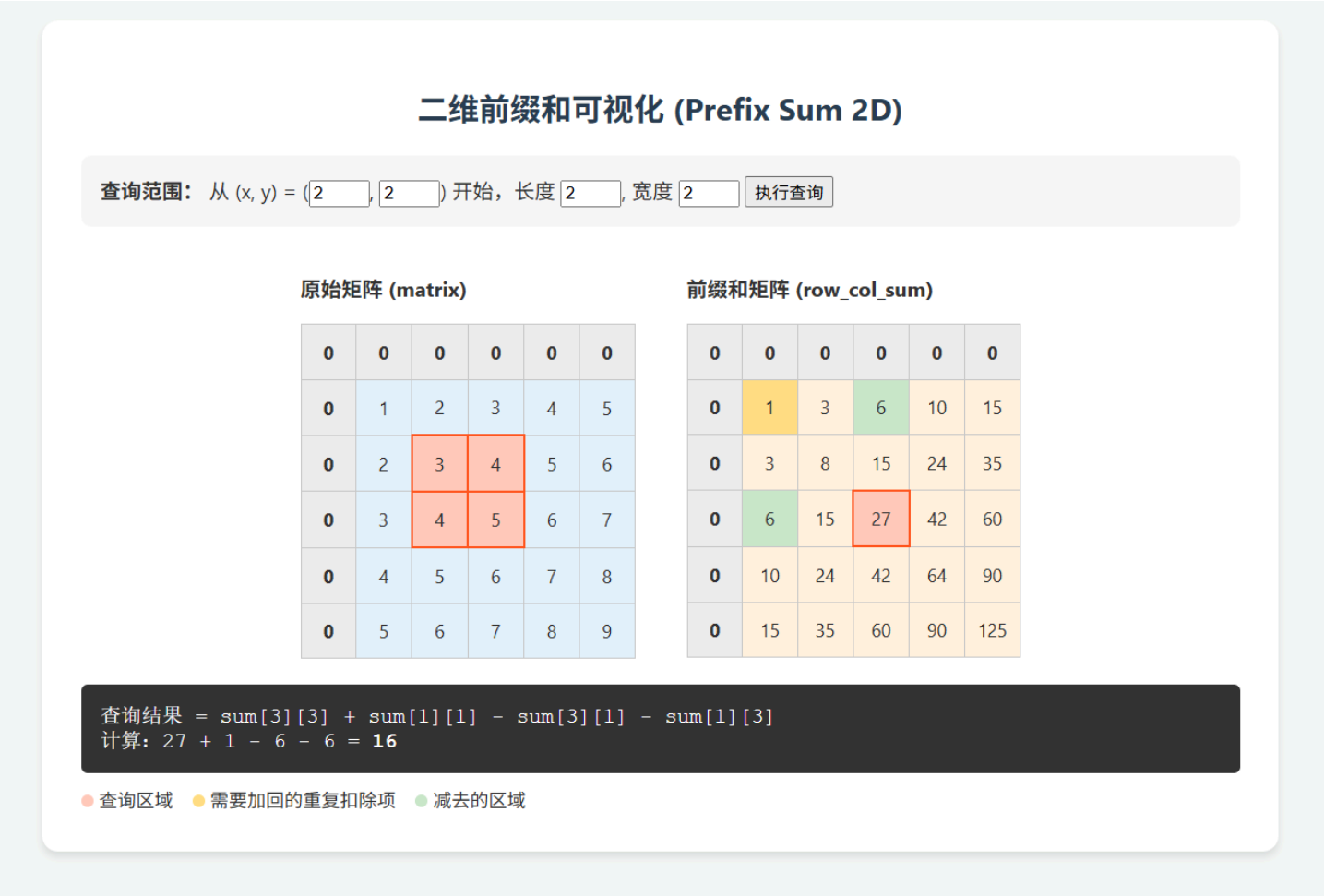
./s3 < input.txt > output.txt 0.00s user 0.00s system 81% cpu 0.007 total

4D

在 m, n 的规模较小，且 q 的规模较大时，solution_3.cpp 会展现出明显的优势，此时预处理的时间被缩小，而访问快的优势被放大

5. AI 工具使用

5A



代码见报告最后

5B

实验评分：92/100

1. 你提到了数组大小和溢出问题
   - 建议：除了使用 long long，建议在报告中给出一个具体的边界计算公式。例如：最大和约为 $2000 \times 2000 \times 10^5 = 4 \times 10^{11}$，而 int 的上限约为 2×10^9，从而定量证明为何必须使用 long long
2. 你在 4C 中对比了三个程序的运行时间
   - 建议将 4C 的测试数据绘制成折线图。观察随着 q 或 a, b 增大，s1 (暴力) 和 s3 (前缀和) 的耗时增长曲线，这比单纯的数字对比更具冲击力。
3. 你的 HTML 可视化已经很棒了。
   - 进阶建议：增加一个“逐步生成”按钮。展示 row_col_sum[i][j] 是如何根据左边、上边和左上角的格子动态填满的。

```html
<!DOCTYPE html>
<html lang="zh-CN">
<head>
    <meta charset="UTF-8">
    <title>二维前缀和算法可视化</title>
    <style>
        body { font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif; background: #f4f7f6; padding: 20px; color: #333; }
        .container { max-width: 900px; margin: 0 auto; background: white; padding: 30px; border-radius: 12px; box-shadow: 0 4px 6px
        h2 { color: #2c3e50; text-align: center; }
        .grid-container { display: flex; gap: 40px; justify-content: center; margin-top: 20px; }
        table { border-collapse: collapse; margin-bottom: 10px; }
        td { width: 40px; height: 40px; border: 1px solid #ccc; text-align: center; font-size: 14px; transition: all 0.3s; }
        .header-cell { background: #eee; font-weight: bold; }

        /* 颜色标识 */
        .origin { background-color: #e3f2fd; } /* 基础矩阵 */
        .prefix { background-color: #fff3e0; } /* 前缀和矩阵 */
        .highlight-rect { background-color: #ffccbc !important; border: 2px solid #ff5722; }
        .highlight-sub { background-color: #c8e6c9 !important; }
        .highlight-add { background-color: #ffe082 !important; }

        .controls { margin: 20px 0; padding: 15px; background: #f9f9f9; border-radius: 8px; }
        .formula { font-family: 'Courier New', Courier, monospace; background: #333; color: #fff; padding: 15px; border-radius: 5px;
        .legend { font-size: 0.9em; margin-top: 10px; display: flex; gap: 15px; }
        .dot { height: 10px; width: 10px; border-radius: 50%; display: inline-block; }
    </style>
</head>
<body>

<div class="container">
    <h2>二维前缀和可视化 (Prefix Sum 2D)</h2>

    <div class="controls">
        <strong>查询范围：</strong>
        从 (x, y) = (<input type="number" id="qx" value="2" min="1" max="4" style="width:40px">,
        <input type="number" id="qy" value="2" min="1" max="4" style="width:40px">)
        开始，长度 <input type="number" id="qa" value="2" min="1" max="3" style="width:40px">,
        宽度 <input type="number" id="qb" value="2" min="1" max="3" style="width:40px">
        <button onclick="visualize()">执行查询</button>
    </div>

    <div class="grid-container">
        <div>
            <p><strong>原始矩阵 (matrix)</strong></p>
            <table id="matrixTable"></table>
        </div>
        <div>
            <p><strong>前缀和矩阵 (row_col_sum)</strong></p>
            <table id="prefixTable"></table>
        </div>
    </div>

    <div class="formula" id="formulaBox">
        点击“执行查询”查看计算逻辑...
    </div>

    <div class="legend">
        <span><span class="dot" style="background:#ffccbc"></span> 查询区域</span>
        <span><span class="dot" style="background:#ffe082"></span> 需要加回的重复扣除项</span>
        <span><span class="dot" style="background:#c8e6c9"></span> 减去的区域</span>
    </div>
</div>

<script>
```

```javascript
const n = 5, m = 5;
const matrix = [
    [0, 0, 0, 0, 0, 0],
    [0, 1, 2, 3, 4, 5],
    [0, 2, 3, 4, 5, 6],
    [0, 3, 4, 5, 6, 7],
    [0, 4, 5, 6, 7, 8],
    [0, 5, 6, 7, 8, 9]
];

const sum = Array.from({ length: n + 1 }, () => Array(m + 1).fill(0));

// 初始化前缀和
for (let i = 1; i <= n; i++) {
    for (let j = 1; j <= m; j++) {
        sum[i][j] = sum[i - 1][j] + sum[i][j - 1] - sum[i - 1][j - 1] + matrix[i][j];
    }
}

function renderTables() {
    const mTable = document.getElementById('matrixTable');
    const pTable = document.getElementById('prefixTable');
    mTable.innerHTML = '';
    pTable.innerHTML = '';

    for (let i = 0; i <= n; i++) {
        let rowM = mTable.insertRow();
        let rowP = pTable.insertRow();
        for (let j = 0; j <= m; j++) {
            let cellM = rowM.insertCell();
            let cellP = rowP.insertCell();

            cellM.innerText = matrix[i][j];
            cellP.innerText = sum[i][j];

            if (i === 0 || j === 0) {
                cellM.className = 'header-cell';
                cellP.className = 'header-cell';
            } else {
                cellM.className = 'origin';
                cellP.className = 'prefix';
            }
            cellM.id = `m-${i}-${j}`;
            cellP.id = `p-${i}-${j}`;
        }
    }
}

function visualize() {
    renderTables(); // 重置样式

    const x = parseInt(document.getElementById('qx').value);
    const y = parseInt(document.getElementById('qy').value);
    const a = parseInt(document.getElementById('qa').value);
    const b = parseInt(document.getElementById('qb').value);

    const x2 = x + a - 1;
    const y2 = y + b - 1;

    if (x2 > n || y2 > m) {
        alert("范围超出矩阵边界！");
        return;
    }
```

```
        // 1. 高亮原始矩阵中的目标区域
        for (let i = x; i <= x2; i++) {
            for (let j = y; j <= y2; j++) {
                document.getElementById(`m-${i}-${j}`).classList.add('highlight-rect');
            }
        }

        // 2. 高亮前缀和矩阵的关键点
        // sum[x2][y2]
        document.getElementById(`p-${x2}-${y2}`).classList.add('highlight-rect');
        // sum[x-1][y-1]
        document.getElementById(`p-${x-1}-${y-1}`).classList.add('highlight-add');
        // sum[x2][y-1] 和 sum[x-1][y2]
        document.getElementById(`p-${x2}-${y-1}`).classList.add('highlight-sub');
        document.getElementById(`p-${x-1}-${y2}`).classList.add('highlight-sub');

        // 3. 更新公式显示
        const result = sum[x2][y2] + sum[x-1][y-1] - sum[x2][y-1] - sum[x-1][y2];
        document.getElementById('formulaBox').innerHTML = `
            查询结果 = sum[${x2}][${y2}] + sum[${x-1}][${y-1}] - sum[${x2}][${y-1}] - sum[${x-1}][${y2}] <br>
            计算: ${sum[x2][y2]} + ${sum[x-1][y-1]} - ${sum[x2][y-1]} - ${sum[x-1][y2]} = <strong>${result}</strong>
        `;
    }

    renderTables();
</script>

</body>
</html>
```